Lecture 2: Pattern Matching & ELIZA

The Fundamentals of Conversational AI 🔧

PSYC 51.07: Models of Language and Communication

Dr. Jeremy R. Manning

Week 1 - Day 2

Today's Journey 

Last Time... 🔄

Key Ideas from Lecture 1

- Consciousness is complex and hard to define
- Language and thought are separable but interactive
- Current LLMs are not conscious
- Pattern matching can create illusions of understanding

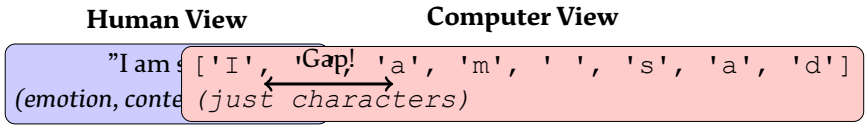
💬 Discussion

Today we'll see *how* pattern matching can create these illusions by building our own simple conversational system!

The Fundamental Challenge 💻

Core Problem

Computers don't "understand" meaning—they manipulate **strings of characters**.



The question: How do we bridge this gap?

Everything Starts with Patterns 🎯

Basic text operations:

1. **Finding** specific words or phrases
2. **Replacing** parts of text
3. **Extracting** information
4. **Transforming** input to output

Important Insight

Even sophisticated models like GPT-4 are (at their core) doing *pattern matching*—just at a much more complex level!

💭 **Think about it!**

If you can master simple patterns, you'll understand the foundations of ALL natural language processing!

Simple String Replacement

Example 1: Direct replacement

```
1 user_input = "I am feeling sad"
2 response = user_input.replace("I am", "Why are you")
3 print(response)
4 # Output: "Why are you feeling sad"
```

What happened here?

- Found the substring ”I am”
- Replaced it with ”Why are you”
- Everything else stayed the same

Think about it!

This seems intelligent! But is it? What are the limitations?

The Problem with Direct Replacement ❌

What if the input is slightly different?

```
# Works
user_input = "I am sad"
response = user_input.replace("I am", "Why are you")
# "Why are you sad" ✓

# Doesn't work
user_input = "I'm sad"
response = user_input.replace("I am", "Why are you")
# "I'm sad" (unchanged!) ✗

# Doesn't work
user_input = "I am very very sad"
response = user_input.replace("I am", "Why are you")
# "Why are you very very sad"
# (But we only wanted to capture "sad") ✗
```

Introduction to Regular Expressions 🎯

What are Regular Expressions?

A powerful language for describing patterns in text, not just exact matches.

Example: Matching with wildcards

```python
import re

pattern = r"I am (.*)"
match = re.search(pattern, "I am feeling sad")

if match:
    feeling = match.group(1)  # Captures "feeling sad"
    response = f"Why are you {feeling}?"
    print(response)
# Output: "Why are you feeling sad?"
```

Regex Syntax Basics 📝

Pattern	Meaning	Example
`.`	Any single character	`c.t` matches ”cat”, ”cut”
`*`	Zero or more of previous	`ca*t` matches ”ct”, ”cat”, ”caat”
`+`	One or more of previous	`ca+t` matches ”cat”, ”caat”
`?`	Zero or one of previous	`ca?t` matches ”ct”, ”cat”
`()`	Capture group	`(cat)` captures ”cat”
`|`	Or	`cat|dog` matches ”cat” or ”dog”
`[]`	Character class	`[abc]` matches ”a”, ”b”, or ”c”
`^`	Start of string	`^cat` matches ”cat” at start
`$`	End of string	`cat$` matches ”cat” at end

Practice Makes Perfect

Regex can be tricky at first, but it’s an essential skill for NLP!

Regex Examples 💡

Example 1: Email addresses

```
pattern = r"\w+@\w+\.\w+"
text = "Email: joe@example.com"
match = re.search(pattern, text)
# Matches: joe@example.com
```

Example 2: Phone numbers

```
pattern = r"\d{3}-\d{3}-\d{4}"
text = "Call 555-123-4567"
match = re.search(pattern, text)
# Matches: 555-123-4567
```

Example 3: Multiple captures

```
pattern = r"I (.*) my (.*)"
text = "I love my family"
match = re.search(pattern, text)
if match:
    verb = match.group(1)    # "love"
    obj = match.group(2)     # "family"
```

Example 4: Optional parts

```
pattern = r"I'?m? (sad|happy)"
# Matches: "I am sad", "I'm sad",
#          "I am happy", "I'm happy"
```

💭 Think about it!

With these patterns, we can handle much more variation in user input!

Capturing and Using Patterns 🎯

The power of capture groups:

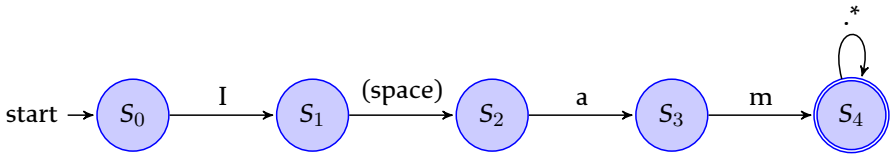
```python
pattern = r"I (.*) my (.*)"
match = re.search(pattern, "I love my family")

if match:
    verb = match.group(1)       # "love"
    object = match.group(2)     # "family"
    response = f"Tell me more about your {object}."
    print(response)
# Output: "Tell me more about your family."
```

💭 Think about it!

This is remarkably simple, yet it can create the *illusion* of understanding! The bot seems to know what you're talking about!

Regex State Machine Visualization



Captures everything after "I am"

How It Works

The regex engine steps through the input character by character, tracking which state it's in. When it matches the pattern, it captures the specified groups.

Pre-Substitutions: Normalizing Input 🔄

Why Pre-Substitutions?

Users type in many different ways. We need to normalize input before pattern matching.

```
pre_subs = {
    "dont": "don't",
    "cant": "can't",
    "wont": "won't",
    "im": "I'm",
    "youre": "you're"
}

user_input = "I dont know"
for old, new in pre_subs.items():
    user_input = user_input.replace(old, new)
```

Post-Substitutions: Fixing Grammar 🔄

Why Post-Substitutions?

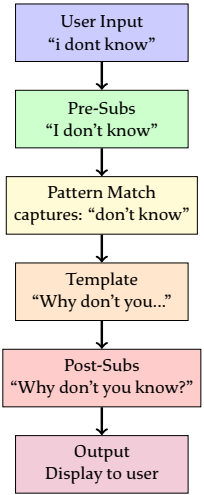
When we mirror user input, we need to flip pronouns and verb forms.

```
post_subs = {
    " i ": " you ",
    " my ": " your ",
    " am ": " are ",
    " was ": " were ",
    " I ": " you "
}

# User said: "I am sad"
# We extracted: "am sad"
# Initial response: "Why are you am sad" (WRONG!)

response = "Why are you am sad"
```

The Substitution Pipeline



ELIZA: A Revolutionary Program 🎭

Weizenbaum (1966)

ELIZA — A Computer Program For the Study of Natural Language Communication Between Man and Machine

What was ELIZA?

- First "chatbot" (1964-1966)
- Simulated a Rogerian psychotherapist
- Used pattern matching & string manipulation
- Created by Joseph Weizenbaum at MIT

💭 Think about it!

ELIZA was meant to *demonstrate* the superficiality of human-computer interaction. Instead, people became emotionally attached to it!

Why a Rogerian Therapist? 🧠

Carl Rogers' Approach

Rogerian therapy is *non-directive*—the therapist mainly reflects what the patient says back to them.

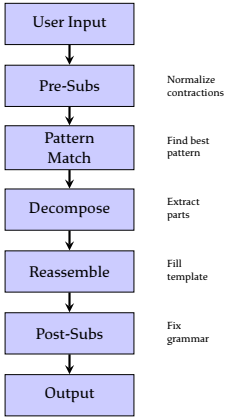
Perfect for a chatbot because:

- ✔️ Doesn't require understanding the problem
- ✔️ Can use simple reflection ("Tell me more about X")
- ✔️ Asks open-ended questions
- ✔️ Minimal need for world knowledge
- ✔️ Pattern matching is sufficient!

💬 Discussion

Patient: I'm sad about my mother.

How ELIZA Works: High Level



No Understanding Required!

ELIZA had **zero** understanding. It's pure pattern transformation!

Pattern Priority and Ranking 🎯

Multiple Pattern Problem

What if more than one pattern matches the input?

Example input: "I am sad about my mother"
Multiple matches:

- Pattern 1: `I am (.*)` - Very general
- Pattern 2: `(.*) mother (.*)` - Specific keyword
- Pattern 3: `I am (.*) about (.*)` - More specific

ELIZA's solution:

1. Each pattern has a **rank** (priority score)
2. Keywords like "mother", "father", "family" get higher ranks
3. More specific patterns generally rank higher
4. Choose the highest-ranking pattern that matches

ELIZA Pattern Structure 📋

A typical ELIZA rule:

```
{
    "pattern": r"(.*) my (mother|father|family) (.*)",
    "rank": 100,  # High priority for family keywords
    "responses": [
        "Tell me more about your {2}.",
        "How do you feel about your {2}?",
        "What role does your {2} play in this?",
        "Does your {2} influence how you feel?"
    ]
}
```

How it works:

1. `pattern`: Regex to match input
2. `rank`: Priority level (higher = more important)
3. `responses`: List of possible response templates
4. `{2}`: Placeholder for captured group 2

Example ELIZA Conversation 💬

Sample Interaction

ELIZA: Welcome. What brings you here today?
User: I am feeling sad.
ELIZA: Why do you feel sad?
User: I can't stop thinking about my family.
ELIZA: Tell me more about your family.
User: My mother never understood me.
ELIZA: Who else in your family never understood you?
User: I have to go now.
ELIZA: That will be $200. See you again next week.

Breaking Down the Conversation 🔍

Let's analyze what happened:

1. **User:** "I am feeling sad"
   - Pattern: `I am (.*)`
   - Captures: "feeling sad"
   - Response template: "Why do you {1}?"
2. **User:** "I can't stop thinking about my family"
   - Pattern: `(.*) my family (.*)`
   - Higher rank due to keyword "family"
   - Response: "Tell me more about your family."
3. **User:** "My mother never understood me"
   - Pattern: `my (mother|father) (.*) me`
   - Response: "Who else in your family {2} you?"
   - Post-sub: "me" stays "you"

Why Does This Work? 🤔

The Psychology Behind ELIZA

Humans are pattern-seeking creatures who project understanding onto machines.

Factors that create the illusion:

1. **Context**: We assume the bot understands context
2. **Expectations**: We expect intelligence from human-like responses
3. **Language structure**: Natural language has inherent patterns
4. **Confirmation bias**: We remember hits, forget misses
5. **Anthropomorphization**: We attribute human qualities to machines

💭 Think about it!

The ”intelligence” is often in the *interpreter* (you!), not the program!

When Pattern Matching Fails 💥

Example Failure Cases

User: My mother and father are fighting.
ELIZA: Tell me more about your father. *(Only caught "father")*

User: I saw a mother duck with her babies.
ELIZA: Tell me more about your mother. *(Wrong context!)*

User: Why do you keep asking about my family?
ELIZA: Tell me more about your family. *(Recursive failure)*

Important Lesson

Pattern matching works surprisingly well, but it fundamentally lacks understanding. It can't handle context, sarcasm, or complex reasoning.

Comparison: ELIZA vs Modern LLMs 📊

Capability	ELIZA (1966)	GPT-4 (2023)
Pattern matching	✓ Simple regex	✓ Neural patterns
Learning	✗ Hand-coded	✓ From data
Context	✗ None	✓ Long-range
World knowledge	✗ None	✓ Extensive
Semantic understanding	✗ None	❓ Debated
Consciousness	✗ None	✗ None

💭 Think about it!

Both lack consciousness, but GPT-4 is vastly more sophisticated. Does this difference in degree create a difference in kind?

What You've Learned Today 🎓

1. **String manipulation**: Basic text transformations
2. **Regular expressions**: Flexible pattern matching
3. **Capture groups**: Extracting information from patterns
4. **Pre/post substitutions**: Normalizing input and output
5. **ELIZA architecture**: How a simple chatbot works
6. **Pattern ranking**: Choosing between multiple matches
7. **Limitations**: Where pattern matching breaks down

The Foundation

These simple techniques form the foundation for all NLP systems, even modern ones!

Next Lecture: Deep Dive into ELIZA 🔮

Lecture 3: ELIZA Implementation & The ELIZA Effect

We'll cover:

- Complete implementation details
- Synonym substitutions
- Memory and context (primitive state)
- The ELIZA Effect and its implications
- Weizenbaum's warnings
- Assignment 1: Building your own ELIZA

To prepare:

- Read Weizenbaum (1966) - Focus on Section 3
- Practice regex with online tools (regex101.com)
- Think about conversation patterns

Practice Exercises 💪

Try These Before Next Class

1. Write a regex to match: ”I think about X” where X can be anything
2. Create pre-substitutions for: ”gonna”, ”wanna”, ”gotta”
3. Design a response template for: ”I hate my X”
4. Think of 3 patterns a therapist might use
5. Find cases where simple pattern matching would fail

Hint

Try your patterns on regex101.com - it has excellent visualization and explanations!

Key Takeaways 🔑

1. Pattern matching can create **convincing illusions** of understanding
2. Regular expressions are **essential tools** for NLP
3. ELIZA was **revolutionary** despite being simple
4. The **gap between behavior and understanding** is crucial
5. Even modern AI fundamentally relies on **pattern matching**
6. Understanding the basics helps you **think critically** about AI

💬 Discussion

After today, when you interact with ChatGPT or other AI, can you spot the "ELIZA moments"—where it's clearly pattern matching without understanding?

Questions? 🤔

Let's discuss!

Contact Information:

📧 jeremy@dartmouth.edu

💬 Discord: https://discord.gg/sftEk9Ygdw

🏢 Office Hours: By appointment (Moore Hall 349)

See you next class! 🚀